ATIVIDADE ACADÊMICA — AULA 05 — AUDITORIA DE SEGURANÇA

Relatório de Auditoria de Segurança — FastHotel

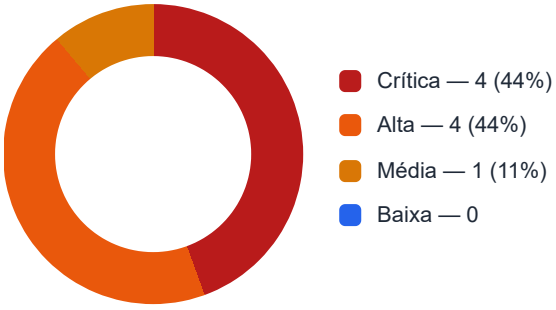
Análise estática do código real do sistema FastHotel (backend Node.js/Express/PostgreSQL e frontend React/TypeScript)

Projeto	FastHotel — github.com/MarcusGiordani/FastHotelTCC
Data	02 de setembro de 2026
Escopo	fasthotel-api/ (backend), fasthotel-app/ (frontend), .github/workflows/ (CI), database/schema.sql
Stack identificada	Node.js + Express + PostgreSQL (pg) · React 19 + TypeScript (CRA) · JWT + bcryptjs · express-validator
Metodologia	Revisão manual do código-fonte real (rotas, controllers, middlewares, componentes de frontend), sem execução de scanners automáticos. Cada achado exige evidência concreta em arquivo e linha. Nenhuma correção foi aplicada ao código do sistema.

2. Resumo Executivo

9	4	4	1
ACHADOS TOTAIS	CRÍTICOS	ALTOS	MÉDIOS

Achados por severidade



Nenhum achado de severidade Baixa ou Informativa foi identificado; os valores acima refletem exatamente a contagem real dos 9 achados (seção 6).

Achados por categoria



Categorias 1, 4 e 5 foram integralmente analisadas e não apresentaram achado verificável (barras em verde) — ver seção 7. A causa raiz da Categoria 1 está tecnicamente refletida nos achados F07/F08 da Categoria 3 (ver seção 6 e seção 7).

> **Achado mais crítico:** a rota pública de cadastro (`POST /api/usuarios`) permite que qualquer visitante, sem autenticação, crie uma conta com `tipo_usuario: "admin"` — e, ainda que essa rota fosse corrigida, um usuário comum autenticado pode se auto-promover a administrador via `PUT /api/usuarios/:id`, pois esse endpoint não verifica propriedade nem restringe o campo `tipo_usuario` (achados F01 e F02).

3. Stack Detectada

Confirmada por leitura direta de `package.json`, código-fonte e arquivos de configuração — nenhuma tecnologia foi assumida sem evidência.

TECNOLOGIA	UTILIZAÇÃO
Node.js + Express 4	Backend REST (`fasthotel-api/server.js`, `fasthotel-api/app.js`)
PostgreSQL (driver pg)	Banco relacional, acesso via SQL parametrizado direto (`fasthotel-api/config/db.js`); sem ORM/query builder
React 19 + TypeScript + react-scripts (CRA)	Frontend (`fasthotel-app/src`)
react-router-dom v7	Roteamento client-side (`fasthotel-app/src/routes/index.tsx`)
styled-components	Estilização dos componentes React
jsonwebtoken (JWT)	Autenticação stateless — token assinado com `process.env.JWT_SECRET`, expiração de 1h (`userController.js`)
bcryptjs	Hash de senha (`genSalt(10)` + `hash`/`compare`)
Middleware `authMiddleware.js`	Valida o JWT do header `x-auth-token` e popula `req.user`
Middleware `authorize.js`	Restringe rota por `tipo_usuario` (`admin`/`recepcionista`/`cliente`) — usado de forma inconsistente entre rotas (ver seção 6)
express-validator	Validação/normalização de entrada (`isEmail`, `normalizeEmail`, `isLength`, `isIn`) nas rotas de usuário e hóspede
Biblioteca de sanitização de HTML	Não identificada e não necessária — não há sink de HTML controlado pelo usuário no frontend (ver seção 7)
Socket.IO	Chat em tempo real (`server.js`, `ChatOnline`)
Jest + Supertest	Testes unitários e de integração do backend (`fasthotel-api/tests`)
Playwright	Testes E2E (`fasthotel-app/playwright.config.ts`, `fasthotel-app/tests/e2e`)
GitHub Actions	CI (`.github/workflows/ci.yml`) — roda unitários, integração, E2E e build a cada push/PR para `main`
Docker	**Não utilizado** — nenhum `Dockerfile`/`docker-compose` encontrado no repositório
Arquivos `.env`	`fasthotel-api/.env`, `fasthotel-api/.env.test` (reais, git-ignorados) e `fasthotel-api/.env.example`, `fasthotel-api/.env.test.example`, `fasthotel-app/.env` (config local, não sensível)

4. Pontos Fortes

Proteções e boas práticas efetivamente verificadas no código — evidência de que a auditoria percorreu o sistema além de procurar apenas problemas.

- OK **Senhas com hash bcrypt:** `userController.js` (`createUser`, `changePassword`) usa `bcrypt.genSalt(10)` + `bcrypt.hash`; nenhuma senha em texto puro é gravada ou retornada pela API.
- OK **Senha nunca retornada:** todos os `SELECT`/`RETURNING` de usuário (`createUser`, `getAllUsers`, `getUserById`, `updateUser`, `loginUser`) listam colunas explicitamente e nunca incluem `senha` no JSON de resposta.
- OK **Troca de senha corretamente escopada ao próprio usuário:** `changePassword` (`userController.js:172-204`) usa `req.user.id` (do token) para localizar o registro, nunca um `id` vindo da URL/body — impossível alterar a senha de outra conta por esse endpoint.
- OK **Reservas isolam corretamente por usuário:** `bookingController.getAllBookings` e `getBookingById` (linhas 111-192) filtram por `usuario_id = req.user.id` quando o solicitante não é `admin`; `updateBooking`/`deleteBooking` verificam `tipo_usuario` antes de alterar/excluir, redundante com a proteção de rota.
- OK **Relatórios/analytics corretamente restritos:** todas as 8 rotas de `reportRoutes.js` exigem `auth` + `authorize(['admin','recepcionista'])`, incluindo o dashboard e os relatórios agregados.
- OK **SQL parametrizado em 100% das queries de aplicação:** nenhuma concatenação de entrada do usuário em string SQL foi encontrada em nenhum controller — todas usam `$1, $2, ...`.
- OK **CORS restrito:** `fasthotel-api/app.js` configura `origin: process.env.FRONTEND_URL` (não usa `*`).
- OK **Nenhum segredo real no repositório:** busca completa no código, configs, CI e histórico do Git não encontrou chave, token ou senha real — apenas placeholders e credenciais de teste (ver seção 7).
- OK **Isolamento do banco de testes:** os testes de integração e E2E têm trava explícita que recusa executar se `PGDATABASE` não for `fasthotel_test` (`tests/integration/setup.env.js`), nunca tocando o banco de desenvolvimento `fasthotel_db`.
- OK **Sem sink de HTML perigoso no frontend:** nenhum uso de `dangerouslySetInnerHTML`, `innerHTML`, `eval` ou `new Function` em toda a árvore `fasthotel-app/src`.
- OK **JWT com expiração curta:** tokens assinados com `expiresIn: '1h'` (`userController.js:149`), reduzindo a janela de uso de um token roubado.

5. Pontos Fracos

Riscos realmente encontrados no código, listados aqui em resumo. O detalhamento técnico completo está na seção 6.

- **Crítica** Qualquer visitante não autenticado pode se autocadastrar como `admin` (F01).
- **Crítica** Qualquer usuário autenticado pode se autopromover a `admin` ou alterar dados de outra conta (F02).
- **Média** Qualquer usuário autenticado pode consultar dados de qualquer outro usuário por ID (F03).
- **Crítica** Gestão de quartos (criar/editar/excluir) não exige nenhuma autenticação (F04).
- **Alta** Qualquer usuário autenticado pode ler/editar/excluir qualquer hóspede, incluindo CPF e RG (F05).
- **Alta** Qualquer usuário autenticado pode alterar o catálogo de serviços/preços (F06).
- **Alta** Qualquer usuário autenticado pode lançar/excluir consumos em reservas de terceiros (F07).
- **Alta** Qualquer usuário autenticado pode ler pagamentos de qualquer reserva (F08).
- **Crítica** Qualquer usuário autenticado pode registrar um pagamento "aprovado" para qualquer reserva sem validação (F09).

6. Achados Detalhados

ID	CATEGORIA	SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
F01	2 — Permissão no navegador	Crítica	userRoutes.js:9-18	Autocadastro público permite criar conta admin
F02	3 — IDOR	Crítica	userController.js:73-96	PUT /usuarios/:id sem ownership permite autopromoção a admin
F03	3 — IDOR	Média	userController.js:58-70	GET /usuarios/:id expõe dados de qualquer usuário
F04	2 — Permissão no navegador	Crítica	roomRoutes.js:7-11	CRUD de quartos sem nenhuma autenticação
F05	3 — IDOR	Alta	guestRoutes.js:7-11	CRUD de hóspedes sem authorize/ownership (PII)
F06	3 — IDOR	Alta	serviceRoutes.js:8-12	CRUD de serviços sem authorize
F07	3 — IDOR	Alta	consumptionRoutes.js:8-12	CRUD de consumos sem authorize/ownership
F08	3 — IDOR	Alta	paymentController.js:29-114	Leitura de pagamentos sem ownership
F09	3 — IDOR	Crítica	paymentController.js:5-26	Criação de pagamento de terceiro, aprovado por padrão

F01 — Autocadastro público permite criar conta administradora

Categoria: 2 — Permissão definida no navegador | **Severidade:** Crítica | **Rota:** POST /api/usuarios

Arquivos: `fasthotel-api/routes/userRoutes.js:9-18`; `fasthotel-app/src/pages/UserRegistration/index.tsx:22,41,81-90`; `fasthotel-app/src/routes/index.tsx:24`

```
// fasthotel-api/routes/userRoutes.js:9-18
router.post(
    '/',
    [ body('nome')..., body('email')..., body('senha')...,
      body('tipo_usuario').optional().isIn(['admin', 'recepcionista', 'cliente'])... ],
    userController.createUser   // <- SEM middleware "auth" antes deste handler
);
```

```
// fasthotel-app/src/pages/UserRegistration/index.tsx:87-89
<option value="recepcionista">Recepcionista</option>
<option value="admin">Administrador</option>
<option value="cliente">Cliente / Hóspede</option>
```

Por que é explorável: a rota de cadastro não possui middleware de autenticação e aceita o campo `tipo_usuario` diretamente do corpo da requisição; a tela pública `/register` (fora de `PrivateRoute`) oferece livremente a opção "Administrador" no formulário.

Impacto: comprometimento total do sistema — qualquer pessoa na internet obtém uma conta `admin` sem qualquer credencial prévia.

Condições de exploração: nenhuma. Não exige autenticação, não exige conhecimento prévio do sistema.

Evidência: `POST /api/usuarios` com corpo `{"nome":"x","email":"x@x.com","senha":"123456","tipo_usuario":"admin"}` retorna `201` e cria a conta como administrador, sem qualquer cabeçalho de autenticação.

F02 — Auto-promoção a administrador via PUT /api/usuarios/:id

Categoria: 3 — IDOR / Escalonamento de privilégio | **Severidade:** Crítica | **Rota:** PUT /api/usuarios/:id

Arquivos: `fasthotel-api/routes/userRoutes.js:22`; `fasthotel-api/controllers/userController.js:73-96`

```
// fasthotel-api/routes/userRoutes.js:22
router.put('/:id', auth, authorize(['admin', 'recepcionista', 'cliente']), userController.updateUser);
// authorize() lista TODOS os papéis válidos -> equivale a não restringir nada

// fasthotel-api/controllers/userController.js:73-81
const updateUser = async (req, res) => {
    const { id } = req.params;
    const { nome, email, tipo_usuario } = req.body;
    const updatedUser = await pool.query(
        'UPDATE usuarios SET nome = $1, email = $2, tipo_usuario = $3 WHERE id = $4 RETURNING id, nome, email,
tipo_usuario',
        [nome, email, tipo_usuario, id]     // <- "id" da URL, sem comparar com req.user.id
    );
```

Por que é explorável: o handler nunca compara `req.params.id` com `req.user.id`, e grava qualquer valor de `tipo_usuario` enviado no corpo, sem restringir esse campo a quem já é `admin`.

Impacto: qualquer conta `cliente` chama `PUT /api/usuarios/<próprio id>` com `{"tipo_usuario":"admin"}` e se torna administrador; o mesmo endpoint permite alterar e-mail/nome/papel de **qualquer outro** usuário, inclusive rebaixar administradores existentes.

Condições de exploração: exige apenas login válido como qualquer papel (inclusive o de menor privilégio, `cliente`).

Evidência: com token JWT de um usuário `cliente`, `PUT /api/usuarios/<id do próprio token>` com `{"tipo_usuario":"admin"}` retorna `200` e o campo `tipo_usuario` da resposta passa a ser `"admin"`.

F03 — Exposição de dados de qualquer usuário via GET /api/usuarios/:id

Categoria: 3 — IDOR | **Severidade:** Média | **Rota:** GET /api/usuarios/:id

Arquivos: `fasthotel-api/routes/userRoutes.js:21`; `fasthotel-api/controllers/userController.js:58-70`

```
// fasthotel-api/controllers/userController.js:58-61
const getUserById = async (req, res) => {
    const { id } = req.params;
    const user = await pool.query(
      'SELECT id, nome, email, tipo_usuario, data_cadastro FROM usuarios WHERE id = $1', [id]
    );  // <- nenhuma comparação com req.user.id nem restrição de papel
```

Por que é explorável: a consulta usa apenas o `id` da URL; a rota permite os três papéis válidos (`admin`, `recepcionista`, `cliente`), tornando a checagem de `authorize` inócua.

Impacto: enumeração de contas (nome, e-mail, papel) de qualquer usuário do sistema por qualquer conta autenticada; não expõe senha (a query nunca seleciona essa coluna).

Condições de exploração: exige apenas login válido, qualquer papel; exige conhecer/adivinhar um ID numérico (sequencial).

Evidência: com token de um `cliente`, `GET /api/usuarios/1` (ou qualquer outro ID) retorna `200` com nome/e-mail/papel do usuário de ID 1, independentemente de quem fez a chamada.

F04 — CRUD de quartos sem nenhuma autenticação

Categoria: 2 — Permissão definida no navegador | **Severidade:** Crítica | **Rotas:** POST/GET/PUT/DELETE /api/quartos[/:id]

Arquivos: `fasthotel-api/routes/roomRoutes.js:7-11`; `fasthotel-api/controllers/roomController.js:5-30,72-95,98-110`

```
// fasthotel-api/routes/roomRoutes.js:7-11 (arquivo completo de rotas)
router.post('/', roomController.createRoom);
router.get('/', roomController.getAllRooms);
router.get('/:id', roomController.getRoomById);
router.put('/:id', roomController.updateRoom);
router.delete('/:id', roomController.deleteRoom);
// nenhuma rota usa "auth" — compare com bookingRoutes.js, que usa auth em todas
```

Por que é explorável: diferentemente de todas as outras entidades do sistema, nenhuma rota de `quartos` exige o middleware `auth`; o controller também não implementa verificação própria.

Impacto: qualquer pessoa na internet, sem login, pode criar quartos falsos, alterar preços/status ou excluir quartos existentes, afetando diretamente a operação e o faturamento do hotel.

Condições de exploração: nenhuma.

Evidência: requisição sem cabeçalho `x-auth-token` para `DELETE /api/quartos/1` retorna `200 {"message":"Quarto excluído com sucesso!"}`.

F05 — CRUD de hóspedes sem authorize/ownership (exposição de PII)

Categoria: 3 — IDOR | **Severidade:** Alta | **Rotas:** GET/PUT/DELETE /api/hospedes/:id

Arquivos: `fasthotel-api/routes/guestRoutes.js:7-11`; `fasthotel-api/controllers/guestController.js:96-108,111-142,145-205`; evidência de frontend em `fasthotel-app/src/pages/Guests/index.tsx:218-229` e `fasthotel-app/src/components/PrivateRoute/index.tsx:8-11`

```
// fasthotel-api/routes/guestRoutes.js:7-11
router.post('/', auth, guestController.createGuest);
router.get('/', auth, guestController.getAllGuests);
router.get('/:id', auth, guestController.getGuestById);
router.put('/:id', auth, guestController.updateGuest);
router.delete('/:id', auth, guestController.deleteGuest);
// só "auth" (login válido) - sem authorize(['admin','recepcionista']) e sem checagem de propriedade

// fasthotel-app/src/components/PrivateRoute/index.tsx:8-11
const token = localStorage.getItem('token');
return token ? children : <Navigate to="/" replace />;
// só verifica se existe token - não verifica tipo_usuario
```

Por que é explorável: qualquer usuário autenticado, inclusive `cliente`, chega à tela `/guests` (o componente de rota protegida só olha para o token) e, mesmo sem UI, pode chamar diretamente `GET/PUT/DELETE /api/hospedes/:id` para qualquer hóspede.

Impacto: leitura e alteração de CPF, RG, data de nascimento, endereço e telefone de qualquer hóspede; exclusão de um hóspede remove em cascata suas reservas, pagamentos e conversas de chat (`guestController.js:145-205`).

Condições de exploração: exige apenas login válido, qualquer papel.

Evidência: com token de `cliente`, `GET /api/hospedes/<qualquer id>` retorna `200` com CPF/RG completos do hóspede; `DELETE /api/hospedes/<id>` retorna `200` e remove o registro.

F06 — CRUD de serviços sem authorize

Categoria: 3 — IDOR | **Severidade:** Alta | **Rotas:** POST/PUT/DELETE /api/servicos[/:id]

Arquivos: `fasthotel-api/routes/serviceRoutes.js:8-12`; `fasthotel-api/controllers/serviceController.js:5-21,50-69,72-84`

```
// fasthotel-api/routes/serviceRoutes.js:8-12
router.post('/', auth, serviceController.createService);
router.put('/:id', auth, serviceController.updateService);
router.delete('/:id', auth, serviceController.deleteService);
// só "auth" - qualquer papel autenticado altera o catálogo
```

Por que é explorável: não há `authorize` nas rotas de escrita; o controller também não valida `req.user.tipo_usuario`.

Impacto: um `cliente` pode alterar preços do catálogo de serviços, excluir itens ou criar serviços falsos, afetando faturamento e relatórios financeiros.

Condições de exploração: exige apenas login válido, qualquer papel.

Evidência: com token de `cliente`, `PUT /api/servicos/1` com `{"preco":0}` retorna `200` e altera o preço do serviço para qualquer usuário do sistema.

F07 — CRUD de consumos sem authorize nem verificação da reserva

Categoria: 3 — IDOR | **Severidade:** Alta | **Rotas:** POST/GET/PUT/DELETE /api/consumos[/:id]

Arquivos: `fasthotel-api/routes/consumptionRoutes.js:8-12`; `fasthotel-api/controllers/consumptionController.js:5-34,37-70,105-150,153-165`

```
// fasthotel-api/controllers/consumptionController.js:5-13
const createConsumption = async (req, res) => {
    const { reserva_id, servico_id, quantidade } = req.body;
    // reserva_id vem do body e não é validado contra req.user.id em nenhum momento
```

Por que é explorável: o mecanismo de isolamento por `usuario_id` que existe em `reservas` (ver `bookingController`) não foi propagado para `consumos`; a rota só exige `auth`, não `authorize`, e o controller não confere a quem pertence a `reserva_id`.

Impacto: um `cliente` pode lançar consumos falsos na conta de outro hóspede/reserva, excluir consumos legítimos de terceiros (fraude/sabotagem de faturamento), ou visualizar o consumo de qualquer reserva.

Condições de exploração: exige apenas login válido, qualquer papel; exige conhecer/adivinhar um `reserva_id`.

Evidência: com token de `cliente` A, `POST /api/consumos` com `reserva_id` pertencente ao `cliente` B retorna `201` e o consumo passa a constar na conta de B.

F08 — Leitura de pagamentos sem controle de acesso

Categoria: 3 — IDOR / 1 — Banco sem tranca | **Severidade:** Alta | **Rotas:** GET /api/pagamentos, GET /api/pagamentos/:id

Arquivos: `fasthotel-api/routes/paymentRoutes.js:9,14`; `fasthotel-api/controllers/paymentController.js:29-74,77-114`

```
// fasthotel-api/routes/paymentRoutes.js:9,14
router.get('/', auth, paymentController.getAllPayments);      // sem authorize, sem filtro por dono
router.get('/:id', auth, paymentController.getPaymentById);   // idem

// paymentController.js:29-33 — getAllPayments monta a query e só filtra por reserva_id
// OPCIONAL vindo da query string; nunca por req.user.id
```

Por que é explorável: assim como em F07, o filtro manual por `usuario_id` usado em `bookingController` não foi replicado aqui — a query já faz `JOIN` com `reservas r` (que tem `usuario_id`), mas nunca adiciona a condição `r.usuario_id = req.user.id`.

Impacto: exposição do histórico financeiro (valor pago, método, status, nome do hóspede) de qualquer reserva para qualquer conta autenticada.

Condições de exploração: exige apenas login válido, qualquer papel.

Evidência: com token de `cliente`, `GET /api/pagamentos` retorna a lista completa de pagamentos de **todos** os usuários do sistema, não apenas os próprios.

F09 — Criação de pagamento para reserva de terceiro, aprovado por padrão

Categoria: 3 — IDOR | **Severidade:** Crítica | **Rota:** POST /api/pagamentos

Arquivos: `fasthotel-api/routes/paymentRoutes.js:8`; `fasthotel-api/controllers/paymentController.js:5-26`

```
// fasthotel-api/controllers/paymentController.js:5-17
const createPayment = async (req, res) => {
    const { reserva_id, valor_pago, metodo_pagamento, status_pagamento } = req.body;
    const bookingExists = await pool.query('SELECT id FROM reservas WHERE id = $1', [reserva_id]);
    // só confere se a reserva EXISTE - não confere se pertence a quem chama
    ...
    const newPayment = await pool.query(
        'INSERT INTO pagamentos (reserva_id, valor_pago, metodo_pagamento, status_pagamento) VALUES ($1, $2, $3,
$4) RETURNING *',
        [reserva_id, valor_pago, metodo_pagamento, status_pagamento || 'aprovado']  // <- padrão "aprovado"
    );
```

Por que é explorável: a rota só exige `auth` (nenhum `authorize`); aceita qualquer `reserva_id` existente, de qualquer usuário; e se o chamador não enviar `status_pagamento`, o valor grava automaticamente como `'aprovado'`, sem qualquer confirmação de que um pagamento real ocorreu.

Impacto: fraude financeira — uma conta `cliente` registra a própria hospedagem (ou a de outra pessoa) como paga sem ter pago nada; extratos e relatórios (`getPaymentExtract`, `getReservationPaymentSummaries`) passam a considerar a reserva "Paga".

Condições de exploração: exige apenas login válido, qualquer papel; exige conhecer um `reserva_id` válido (o próprio, por exemplo).

Evidência: com token de `cliente`, `POST /api/pagamentos` com `{"reserva_id": <própria reserva>, "valor_pago": 999999, "metodo_pagamento": "pix"}` (sem `status_pagamento`) retorna `201` com `status_pagamento: "aprovado"`.

7. Categorias Sem Vulnerabilidade Verificável

Categoria 4 — Chaves expostas / segredos

Analisada — sem achado

Foi feita busca no código-fonte completo (backend e frontend), em `.env.example`, `.env.test.example`, no workflow de CI (`.github/workflows/ci.yml`) e em **todo o histórico do Git** (`git log --all -p`) por padrões de chave de API, JWT secret, senha, token de webhook e credenciais em URLs de conexão. Os únicos valores encontrados foram placeholders explícitos (`troque-por-sua-senha-local`, `troque-por-uma-chave-longa-e-aleatoria`, `sua-senha-local`) e uma credencial de teste sem valor sensível (`JWT_SECRET = [REDACTED]`, valor fixo usado apenas nos testes de integração/E2E locais, nunca em produção). O `.env` e o `.env.test` reais (que contêm a senha do PostgreSQL local) nunca foram commitados — confirmado tanto pelo `.gitignore` atual quanto pela ausência desses arquivos em todo o histórico. O CI usa exclusivamente credenciais de teste definidas dentro do próprio arquivo de workflow (`postgres/postgres` no container do PostgreSQL, `JWT_SECRET = [REDACTED]`). **Não foi encontrado achado verificável nesta categoria.**

Categoria 5 — Inputs sem tratamento / XSS

Analisada — sem achado

Foi feita busca por `dangerouslySetInnerHTML`, `innerHTML`, `document.write`, `eval`, `new Function`, `javascript:` e `insertAdjacentHTML` em toda a árvore `fasthotel-app/src` — nenhuma ocorrência. Nenhum componente insere HTML/Markdown vindo do usuário; todo conteúdo dinâmico (incluindo mensagens de chat em `ChatOnline/index.tsx`, que é o dado mais "livre" gerado por usuário no sistema) é renderizado via interpolação JSX padrão (`{msg.conteudo}`), que o React escapa automaticamente. Nenhum `href`/`src` do projeto é construído a partir de entrada do usuário. No backend, não existe geração de e-mail, template HTML ou qualquer resposta que misture HTML com dados do usuário — a API só retorna JSON. **Não foi encontrado achado verificável nesta categoria.**

Categoria 1 — Banco sem tranca (isolamento de dados)

Analisada — mecanismo identificado

O FastHotel não é multi-tenant; o mecanismo de isolamento identificado é um **filtro manual por `usuario_id`**, aplicado quando o solicitante não é `admin`. Esse mecanismo está **corretamente implementado** em `bookingController.getAllBookings` e `getBookingById` (`fasthotel-api/controllers/bookingController.js:111-192`) — ver seção 4, Pontos Fortes. Entretanto, o mesmo filtro **não foi propagado** para as tabelas dependentes de `reservas`: `pagamentos` e `consumos`. Essa lacuna já está documentada tecnicamente como F07 e F08 (Categoria 3) na seção 6 — não há, portanto, um achado adicional e distinto de Categoria 1 além do que já foi reportado nesses dois IDs.

8. Recomendações

Priorização baseada exclusivamente nos achados reais desta auditoria — nenhuma prioridade foi criada para achados inexistentes.

P1 — Urgente Corrigir antes de qualquer outra atividade

- **F01** — Exigir autenticação/privilégio de `admin` para criar contas com `tipo_usuario` diferente de `cliente`; a rota pública de cadastro deve sempre forçar `cliente`.
- **F02** — Em `updateUser`, comparar `req.params.id` com `req.user.id` e proibir a alteração de `tipo_usuario` por quem não é `admin`.
- **F04** — Adicionar `auth` + `authorize(['admin','recepcionista'])` nas rotas de escrita de `roomRoutes.js`.
- **F09** — Restringir `POST /api/pagamentos` a `admin`/`recepcionista`.

P2 — Alto Corrigir na sequência, mesmo padrão de causa raiz

- **F05** — Adicionar `authorize(['admin','recepcionista'])` em `guestRoutes.js`.
- **F06** — Adicionar `authorize(['admin','recepcionista'])` nas rotas de escrita de `serviceRoutes.js`.
- **F07** — Adicionar `authorize` nas rotas de escrita de `consumptionRoutes.js` e filtrar leitura por propriedade da reserva.
- **F08** — Filtrar `getAllPayments`/`getPaymentById` por `reservas.usuario_id = req.user.id` quando o papel não for `admin`/`recepcionista`.

P3 — Moderado

- **F03** — Restringir `GET /api/usuarios/:id` de terceiros a `admin`/`recepcionista`; permitir a qualquer usuário apenas o próprio registro.

O detalhamento de como implementar e testar cada correção está em `docs/security-audit/plano-correcao-vulnerabilidades.md`. Nenhuma correção foi aplicada ao código nesta auditoria.

9. Issues para o GitHub

9 achados agrupados em 6 issues por causa raiz semelhante, prontas para copiar em Markdown.

```
--- ISSUE 1 ---

# [Segurança] Autocadastro público permite criar conta administradora (F01)

Labels sugeridas:
security
severity:critical

## Descrição
POST /api/usuarios não exige autenticação e aceita "tipo_usuario" vindo do
corpo da requisição. A tela pública /register oferece livremente a opção
"Administrador" no formulário, sem qualquer restrição.

## Por que é explorável
Não há middleware "auth" na rota de cadastro (fasthotel-api/routes/userRoutes.js:9-18).
Não há verificação de quem está fazendo a chamada antes de gravar tipo_usuario.

## Evidência

fasthotel-api/routes/userRoutes.js:9-18
fasthotel-app/src/pages/UserRegistration/index.tsx:22,41,81-90
fasthotel-app/src/routes/index.tsx:24

```text
router.post('/', [validações...], userController.createUser); // sem auth
```

Impacto
Comprometimento total do sistema por qualquer visitante não autenticado.

Sugestão de correção
A rota pública deve sempre forçar tipo_usuario='cliente'. Criar rota separada
e protegida (auth + authorize(['admin'])) para criação de contas de equipe.

Critérios de aceite
- POST /api/usuarios não autenticado nunca cria conta admin/recepcionista
- Existe rota protegida por admin para criar contas de equipe
- Teste de integração cobre os dois casos acima

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

[Segurança] Qualquer usuário autenticado pode se autopromover a administrador
  ou alterar dados de qualquer conta (F02, F03)

Labels sugeridas:
security
severity:critical

Descrição
PUT /api/usuarios/:id não verifica se o "id" da URL corresponde ao usuário
autenticado, e grava qualquer "tipo_usuario" enviado no corpo. GET /api/usuarios/:id
retorna dados (nome, e-mail, papel) de qualquer usuário para qualquer autenticado.

Por que é explorável
authorize(['admin','recepcionista','cliente']) inclui todos os papéis válidos,
equivalendo a nenhuma restrição. O controller nunca compara req.params.id com
req.user.id (fasthotel-api/controllers/userController.js:58-70,73-96).
```

Evidência

fasthotel-api/routes/userRoutes.js:21,22
fasthotel-api/controllers/userController.js:58-70 (getUserById)
fasthotel-api/controllers/userController.js:73-96 (updateUser)

```text
const updatedUser = await pool.query(
    'UPDATE usuarios SET nome = $1, email = $2, tipo_usuario = $3 WHERE id = $4 ...',
    [nome, email, tipo_usuario, id]
);
```

Impacto
Qualquer conta "cliente" pode se tornar "admin" chamando o próprio ID, ou alterar/
consultar dados de qualquer outra conta.

Sugestão de correção
Comparar req.params.id com req.user.id; só permitir alterar/consultar dados de
terceiros e o campo tipo_usuario quando req.user.tipo_usuario === 'admin'.

Critérios de aceite
- Um usuário não-admin não consegue alterar tipo_usuario (nem o próprio, nem de outros)
- Um usuário não-admin só consulta/altera o próprio registro
- Teste de integração cobre autopromoção, alteração de terceiro e leitura de terceiro

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

[Segurança] Rotas de gestão de quartos sem nenhuma autenticação (F04)

Labels sugeridas:
security
severity:critical

Descrição
Todas as rotas de /api/quartos (criar, listar, obter, atualizar, excluir) estão
registradas sem o middleware de autenticação "auth".

Por que é explorável
fasthotel-api/routes/roomRoutes.js:7-11 não importa nem usa "auth"/"authorize"
em nenhuma das 5 rotas, diferente de todas as outras entidades do sistema.

Evidência

fasthotel-api/routes/roomRoutes.js:7-11

```text
router.post('/', roomController.createRoom);
router.put('/:id', roomController.updateRoom);
router.delete('/:id', roomController.deleteRoom);
```

Impacto
Qualquer pessoa na internet, sem login, pode criar, alterar preços/status ou
excluir quartos do hotel.

Sugestão de correção
Adicionar auth + authorize(['admin','recepcionista']) nas rotas de escrita
(POST, PUT, DELETE). Leitura pode continuar pública ou exigir apenas auth,
conforme regra de negócio.

Critérios de aceite
- POST/PUT/DELETE em /api/quartos sem token retornam 401
- cliente autenticado recebe 403 nessas rotas (se a regra for admin/recepcionista)

```
- admin/recepcionista continuam funcionando normalmente

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

# [Segurança] Rotas de Hóspedes, Serviços e Consumos sem controle de acesso (F05, F06, F07)

Labels sugeridas:
security
severity:high

## Descrição
As rotas de /api/hospedes, /api/servicos e /api/consumos exigem apenas login
válido ("auth"), sem restringir por papel ("authorize") nem verificar
propriedade do recurso. Qualquer conta "cliente" pode ler/alterar/excluir
hóspedes (incluindo CPF/RG), o catálogo de serviços/preços, e consumos de
QUALQUER reserva.

## Por que é explorável
- fasthotel-api/routes/guestRoutes.js:7-11 (só auth)
- fasthotel-api/routes/serviceRoutes.js:8-12 (só auth)
- fasthotel-api/routes/consumptionRoutes.js:8-12 (só auth)
Nenhum dos três controllers valida req.user.tipo_usuario nem propriedade da reserva.
Confirmado também no frontend: fasthotel-app/src/pages/Guests/index.tsx:218-229
renderiza os botões de editar/excluir para qualquer usuário logado, e
fasthotel-app/src/components/PrivateRoute/index.tsx:8-11 só verifica o token,
nunca o papel.

## Evidência

fasthotel-api/controllers/guestController.js:96-108,111-142,145-205
fasthotel-api/controllers/serviceController.js:5-21,50-69,72-84
fasthotel-api/controllers/consumptionController.js:5-34,105-150,153-165

## Impacto
Exposição/alteração de PII de hóspedes; tampering de preços; fraude/sabotagem
de consumos em reservas de terceiros.

## Sugestão de correção
Adicionar authorize(['admin','recepcionista']) nas rotas de escrita das três
entidades; em Consumos, filtrar leitura por propriedade da reserva
(reservas.usuario_id = req.user.id) para papéis não-admin, no mesmo padrão já
usado em bookingController.getAllBookings.

## Critérios de aceite
- cliente autenticado recebe 403 ao criar/alterar/excluir hóspede, serviço ou consumo
- cliente autenticado, ao listar consumos, só vê os das próprias reservas
- admin/recepcionista continuam funcionando normalmente

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

# [Segurança] Pagamentos sem controle de acesso e criação fraudulenta com status "aprovado" (F08, F09)

Labels sugeridas:
security
severity:critical

## Descrição
GET /api/pagamentos e GET /api/pagamentos/:id retornam dados financeiros de
qualquer reserva para qualquer usuário autenticado. POST /api/pagamentos aceita
qualquer reserva_id e grava status_pagamento='aprovado' por padrão quando o
campo não é enviado, sem qualquer confirmação de pagamento real.
```

```
## Por que é explorável
fasthotel-api/routes/paymentRoutes.js:8,9,14 só exigem "auth" nessas três rotas.
O filtro manual por usuario_id, usado corretamente em bookingController, não foi
propagado para pagamentos.

## Evidência

fasthotel-api/controllers/paymentController.js:5-26 (createPayment)
fasthotel-api/controllers/paymentController.js:29-74 (getAllPayments)
fasthotel-api/controllers/paymentController.js:77-114 (getPaymentById)

```text
[reserva_id, valor_pago, metodo_pagamento, status_pagamento || 'aprovado']
```

Impacto
Exposição de histórico financeiro de terceiros; fraude - qualquer "cliente"
pode marcar a própria (ou outra) reserva como paga sem pagar nada.

Sugestão de correção
Restringir POST /api/pagamentos a admin/recepcionista. Filtrar GET (lista e por
ID) por reservas.usuario_id = req.user.id quando o papel não for admin/recepcionista.

Critérios de aceite
- cliente autenticado recebe 403 ao tentar criar um pagamento
- cliente autenticado só vê pagamentos das próprias reservas
- admin/recepcionista continuam funcionando normalmente

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

[Segurança] GET /api/usuarios/:id expõe dados de qualquer usuário (F03)

Labels sugeridas:
security
severity:medium

Descrição
Qualquer usuário autenticado pode consultar nome, e-mail, tipo_usuario e data
de cadastro de qualquer outro usuário, apenas informando o ID na URL.

Por que é explorável
authorize(['admin','recepcionista','cliente']) na rota GET /:id inclui todos
os papéis válidos; o controller não compara o ID da URL com req.user.id.

Evidência

fasthotel-api/routes/userRoutes.js:21
fasthotel-api/controllers/userController.js:58-70

Impacto
Enumeração de contas do sistema (quem é admin, e-mails) por qualquer conta
autenticada, incluindo "cliente".

Sugestão de correção
Permitir consulta de terceiros apenas para admin/recepcionista; para os demais
papéis, permitir apenas o próprio registro (id === req.user.id).

Critérios de aceite
- cliente consultando o próprio ID recebe 200
- cliente consultando ID de outro usuário recebe 403
- admin/recepcionista continuam consultando qualquer ID

--- FIM ISSUE 6 ---
```